

Supplementary Material

A Graph-Based Cluster Dynamics Framework for Irradiated Materials

S1 The Reaction Admissibility Graph: Lineage, Implementation, and Solver Coupling

This section places the reaction admissibility graph (RAG) of Sec. 2 in the wider theory of chemical reaction networks, describes how the graph is represented in software, sets out the three-layer architecture of RadCluster with emphasis on what a user must supply for a new material, and explains how the choice of time integrator follows from the structure of the graph. Throughout, a cluster vertex is the identifier $\sigma = (\chi, n, p, \mathbf{c})$ (polarity, size, population, composition), the graph is $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \nu, k)$, and the master equation is the graph-walking accumulator

$$\frac{dc_a}{dt} = G_a + \sum_{r \in \mathcal{E}} S_{ar} J_r(\mathbf{c}) - \mathcal{D}_a c_a, \quad (\text{S1})$$

where S_{ar} is the stoichiometric coefficient of vertex a in edge r , J_r the edge flux built from the kernel k_r , G_a the cascade source and $\mathcal{D}_a$ the loss to unresolved fixed sinks.

Relation to chemical reaction network theory. A chemical reaction network (CRN) is a triple of species, complexes and reactions; under mass-action kinetics its dynamics is $\dot{\mathbf{x}} = \mathbf{S}\mathbf{J}(\mathbf{x})$ with a constant stoichiometric matrix $\mathbf{S}$ [1–4]. Equation (S1) has exactly this form, so at the level of the mathematical object the RAG is a CRN: vertices are species, each edge is a reaction, and binary edges (coalescence, annihilation) are directed hyperedges with two tail vertices, the representation used for reaction and metabolic networks [5, 6]. Source and sink edges are the inflow and outflow reactions $\emptyset \rightarrow X$ and $X \rightarrow \emptyset$ of an open CRN. The linear invariants of a CRN are the left null vectors of $\mathbf{S}$; in the RAG the signed defect content $q(\sigma) = \chi n$ is one such invariant for every intra-host edge. Cluster dynamics also descends from the population-balance (Smoluchowski) equation [7], which is itself an infinite CRN with a size-indexed species set. In genealogical terms, the RAG is therefore a *descendant* of CRN theory in formalism, a *sibling* of rule-based network generators in method, and an *extension* of both in the physics it encodes. Table S1 makes the correspondence explicit.

The sibling relation is the most useful one for implementation. Rule-based languages such as BioNetGen and Kappa [8, 9] never enumerate species; they declare *rules* over structured molecules and generate reactions on demand. Automatic mechanism generators in combustion, such as RMG [10], organize reactions into *reaction families* with shared templates and *rate rules*. The RAG follows the same design. An **Edge** is a family parameterized over the size axis, its **EdgeClass** is the template, and the kernel library k plays the role of the rate rules. What the RAG adds is a typed state space in which admissibility can be checked structurally: (i) the polarity index χ splits $\mathcal{V}$ into two layers, so that annihilation is the only binary cross-layer class and inter-population transfer is forbidden from crossing layers (enforced in `Edge.__post_init__`); (ii) each of the ten edge classes carries a fixed stoichiometric contract (`EDGE_CLASS_SPEC`) whose sign pattern makes conservation of q a property of the graph rather than of each host’s equations; (iii) populations p encode mobility law and microstructural environment, so that the same size can occupy several physically distinct vertices (e.g., glissile $\frac{1}{2}\langle 111 \rangle$ and sessile $\langle 100 \rangle$ loops).

The extensions also mark where standard CRN results stop applying. The deficiency and complex-balancing theorems [1, 2] assume finite networks with constant mass-action rate coefficients. RAG kernels are size-dependent diffusion-limited capture rates. Emission kernels are tied

Table S1: Correspondence between chemical reaction network (CRN) concepts and their realization in the RAG and in RadCluster.

CRN concept	RAG realization	Code (RadCluster_2_1/py_utils/core)
Species	Vertex $\sigma = (\chi, n, p, \mathbf{c})$	<code>ClusterIdentifier</code> , <code>Population</code> , <code>Polarity</code>
Reaction	Edge r with tuple ν_r	<code>Edge</code> (one <i>family</i> over n)
Reaction template / rule	Edge class (10 classes)	<code>EdgeClass</code> , <code>EDGE_CLASS_SPEC</code>
Rate constant / rate rule	Kernel $k_r(n, n')$	<code>register_kernel</code> (arrays or lazy builders)
Stoichiometric matrix S	S_{ar} , implied by the class contract	per-class routines of <code>GraphWalker</code>
Conservation law	$q = \chi n$ (and He inventory)	structural; checked by δ_{FP} , δ_{He}
Inflow / outflow	<code>SOURCE</code> / <code>SINK</code> classes	cascade spectrum; $\mathcal{D}_a$
Species–reaction graph	Explicit bipartite/hypergraph view	<code>to_networkx</code> (diagnostics only)
Model reduction (lumping)	Bin-moment and He reductions	<code>reductions/bin_moment.py</code> , <code>he_reduction.py</code>
<i>No CRN analog</i>	Polarity layers; population transfer; admissibility checks	<code>Edge.__post_init__</code> , <code>rag.validate</code>

to capture kernels through binding energies (detailed balance at the level of each reversible pair). The fixed-sink term $\mathcal{D}_a c_a$ is a mean-field closure for an unresolved microstructure. The system is open and driven far from equilibrium by the cascade source. Finally, the RAG supports state-space reductions (logarithmic bin moments, helium mean-field and decoupled loadings) that coarsen the graph while keeping the exact edge set, a graph-level analog of lumping in chemical kinetics [11]. We also note a terminological point: the “master equation” of cluster dynamics is the deterministic reaction-rate equation (S1), not the chemical master equation for probability distributions; the latter corresponds to stochastic cluster dynamics [12, 13], which could walk the same edge set.

Graph representation and use of general network software. RadCluster keeps two representations of the same graph. In the *production* representation the RAG is implicit. The container `ReactionAdmissibilityGraph` stores a registry of populations keyed by (polarity, name), a list of `Edge` families and a kernel library, and it checks admissibility when an edge is added: endpoints must be registered, and the polarity rules of the class must hold. No vertex object is ever created. The graph walker applies each family’s kernel vectorized over the size axis, so one declared edge stands for $O(N)$ unary or $O(N^2)$ binary elementary reactions. The EUROFER-97 instantiation declares three populations and 24 edge families (Table S2), which at the production sizes $I = V = 10^3\text{--}10^4$ would expand to $10^7\text{--}10^9$ explicit arcs. A general-purpose graph library is therefore not in the solver path; the solver needs only NumPy arrays (Python) or contiguous `double` buffers (C++).

The *diagnostic* representation materializes the graph with NetworkX [14]. The function `to_networkx` builds a `networkx.MultiDiGraph` truncated at a size cut-off n_{max} . It creates one node per admissible vertex, carrying polarity, population and size attributes, plus two sentinel nodes, `SOURCE` and `SINK`, so that reachability questions have well-defined endpoints. Each edge family is then expanded class by class: growth as $n \rightarrow n+1$ arcs; shrinkage, dissociation and recombination as $n \rightarrow n-1$; inter-population transfer as $(p, n) \rightarrow (p', n)$; and solute trapping as a tagged self-loop, because the composition axis is not expanded. NetworkX has no native hyperedge type. A binary edge is therefore stored as its two tail→head arcs, both flagged `hyperedge=True` and sharing a `hyper_id`, so the hypergraph can be recovered exactly. Mutual annihilation ($n = n'$) points to `SINK`. Every arc carries its edge class, label and kernel name.

Three uses follow. (a) `structural_report` audits a host declaration with library algorithms: weak connectivity; the set of vertices reachable from the two monomer roots (`nx.descendants`), where an unreachable vertex signals a population with no production channel; and vertices with no outgoing arc (sink-only traps). For EUROFER-97 at $n_{\max} = 30$ the report gives 89 vertices and 7078 arcs, a weakly connected graph, no vertex unreachable from the monomers and no dead vertex. Coalescence (3306 arcs) and annihilation (3258) make up 93% of the arcs, which is the quantitative reason for keeping the production graph implicit. (b) `write_graphml` serializes the materialized graph to GraphML for external tools such as Gephi, yEd or Cytoscape. It strips `None`-valued attributes, which the GraphML writer rejects. (c) The RAG figures of the main text (`codes/make_rag_figure.py`, `make_rag_hyperedge_figure.py`) are drawn with Matplotlib from the NetworkX graph produced by the declared RAG, not from a hand-transcribed edge list, so a figure cannot drift from the model. NetworkX is an optional dependency, imported lazily inside these functions. A run that never calls them does not require it.

Three-layer architecture. RadCluster separates *what a cluster-dynamics model is* from *what a particular material admits* and from *how the equations are integrated* (Fig. S1). The separation follows the open-closed principle [15]: Layers 1 and 3 are closed to modification, and a new material is introduced only by extension in Layer 2.

Layer 1, the abstract core (`py_utils/core`), is host-independent and immutable. It defines the vocabulary of the state space (`Polarity`, `Population`, `ClusterIdentifier`); the ten edge classes and their stoichiometric contracts; the RAG container with its admissibility and kernel-registration logic; the `StateLayout` bijection between vertices and the flat ODE vector (discrete, bin-moment, scalar and auxiliary blocks); the reference `GraphWalker`, which assembles Eq. (S1) by dispatching on edge class only; and the graph-level reductions (`BinMomentReduction`, `HeReductionMode`). Nothing in Layer 1 names a material. It is immutable because its contracts are what make conservation structural and what make every host inherit the reductions and diagnostics unchanged. Editing it for one material would silently change every other.

Layer 2, the host declaration (the Python package `materials/<host>` and its compiled mirror under `cpp_utils`), is the only layer written for a new material. It is a declaration, not an integrator: a function (`build_eurofer_rag` for EUROFER-97) that returns the pair (RAG, `StateLayout`), together with the compiled right-hand side installed as a function pointer for Layer 3. The selections a user must make are listed in Table S2 and, in order, are:

1. **Populations per polarity layer.** Choose the physically distinct cluster phases, each with a minimum admissible size $n_{\min}$ and a mobility cut-off (`mobile_max`), and designate the monomer pool of each layer. Populations are warranted when clusters of the same size differ in mobility law, binding energy or sink coupling (bcc: glissile $\frac{1}{2}\langle 111 \rangle$ versus sessile $\langle 100 \rangle$ loops; hcp: $\langle a \rangle$ versus $\langle c \rangle$ components; fcc: Frank loops versus stacking-fault tetrahedra).
2. **Composition axis.** List the resolved non-host species (`gas_species`), e.g., He for fission/fusion steels, or He and H for plasma-facing tungsten. An empty list removes the solute-trapping class entirely.
3. **Edge families.** Map every physical process to one of the ten classes, giving partner and product populations where the class requires them. The core rejects inadmissible choices (cross-layer transfer, same-polarity annihilation, a coalescence product of the wrong polarity). Excluded channels are simply not declared; for EUROFER-97 these are the reactions between two mutually 1D-mobile clusters. Optional channels are switched by input flags (e.g., `LOOP_NETWORK_LOSS`, default off).

Table S2: Layer-2 selections for a host material, with the EUROFER-97 choices of RadCluster_2_1 (`materials/eurofer97/declaration.py`).

Selection	EUROFER-97 instantiation
SIA populations	<code>bulk-111</code> ($n_{\min} = 1$, mobile to i_{mobile} , SIA monomer pool, all cascade SIA production); <code>bulk-100</code> (sessile, $n_{\min} = n_{\text{loop}}$, default 4)
Vacancy populations	<code>bulk</code> ($n_{\min} = 1$, mobile to v_{mobile} , vacancy monomer pool)
Composition axis	He only
Edge families (24)	P1 recombination; P2 cavity growth/shrinkage; P3 loop growth/shrinkage (both SIA populations); P4 fixed sinks (3); P5 thermal emission (3); SIA–SIA and V–V coalescence; V–I annihilation with $\frac{1}{2}\langle 111 \rangle$ and $\langle 100 \rangle$; $\frac{1}{2}\langle 111 \rangle \rightarrow \langle 100 \rangle$ conversion by unary transfer (Dudarev), junction coalescence and absorption growth (Marian); cascade sources (2); P7 trap mutation; P8 radiation re-resolution
Excluded channels	1D/1D reactions between mutually mobile clusters (kernels in a separate supplementary table)
Kernel inputs	$\Omega = a^3/2$; D_n with solute trapping; $\xi(n) = (3n/8\pi)^{1/3}$; bias factors; $E_b(n)$; ρ_d , d_g , precipitates; fission/fusion cascade spectra
State representation	discrete ($N_{\text{eq}} \approx 2I + V$) or bin-moment (linear, $P = 2$); He Case 2 (fission) or Case 1 (fusion); five conservation integrals

4. **Kernel library k .** Register a size-resolved kernel for every edge: diffusivities with solute trapping, capture radii and geometric prefactors, bias factors, binding energies (which fix emission through detailed balance), fixed-sink strengths and the cascade production spectrum. The physics modules (`reaction_rates.py`, `binding_energies.py`, `defect_production.py`) compute these from the three-sheet input workbook. ($\mathcal{V}, \mathcal{E}$) are host-specific, while k is calibration-specific, so re-calibration changes Layer 2 data but not its topology.
5. **State-space representation.** Select discrete or bin-moment size axes (i_{discrete} , I_{bin} , linear dual-basis closure with $P = 2$) and the helium reduction (Case 1 mean-field for fusion, Case 2 decoupled inventory for fission, or explicit/quasi-steady free He). These choices fix the `StateLayout` and hence N_{eq} .
6. **Compiled right-hand side and verification.** Mirror the declared edges in the host’s compiled kernel file (`rate_kernels.cpp`) and verify them against the Python `GraphWalker`, which serves as the executable specification. The development order is: Python walker, then conservation gate ($\delta_{\text{FP}}, \delta_{\text{He}} < 10^{-6}$), then C++ mirror, then calibration.

Layer 3, the numerical engine (`cpp_utils/core` and the Python bridge `cpp_bridge.py`), is host-independent and immutable. It contains the SUNDIALS/CVODE variable-order BDF driver [16, 17], with non-negativity maintained by a post-step concentration floor rather than by solver constraints, which would introduce a Jacobian kink, and the linear-solver menu: dense, banded, SPGMR [18] with Jacobi or Woodbury preconditioning, and KLU [19] with a Curtis–Powell–Reid colored finite-difference Jacobian [20]. It also contains the OpenMP active-window mode, the size-only kernel tables and the Python bridge, which writes the parameter file, launches the solver and parses the binary trajectory. Layer 3 reaches the physics only through the right-hand-side function pointer `UserData.rhs_fn`; the preconditioners probe whatever right-hand side the host installed. It is immutable for the same reason as Layer 1. Its error control, conservation floor and preconditioner algebra have been verified once, and every material inherits that verification.

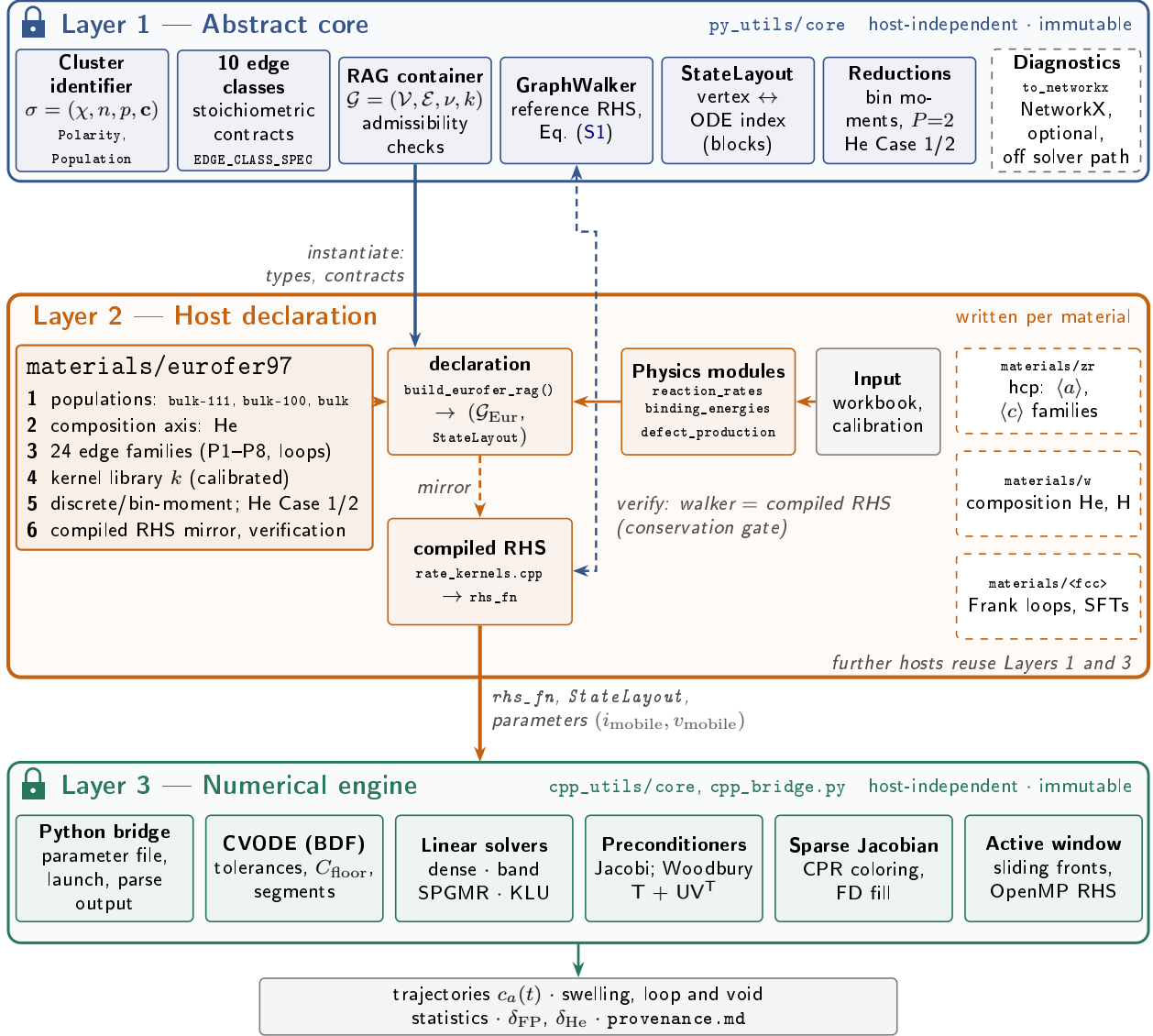


Figure S1: Three-layer architecture of RadCluster. Layer 1 (abstract core) and Layer 3 (numerical engine) are host-independent and closed to modification (padlocks). A material enters only through Layer 2, by declaring populations, the composition axis, edge families, the kernel library and the state-space reduction. Layer 2 hands Layer 3 the **StateLayout** and a compiled right-hand side verified against the Layer 1 graph walker. The NetworkX export is a diagnostic branch outside the solver path. Dashed host cards indicate instantiations that reuse Layers 1 and 3 unchanged.

Relationship between the solver method and the RAG. The RAG is a model; the integrator is a method. The graph fixes the vector field of Eq. (S1) and, through its connectivity, the structure of the Jacobian $J_{ab} = \partial f_a / \partial c_b$. It does not prescribe how the equations are advanced. The same edge set can be walked by the Python reference accumulator, by compiled kernels under CVODE, by the bin-moment “reconstruct–walk–project” wrapper or, in principle, by a stochastic simulation algorithm. The graph does, however, determine which method is efficient, in four ways.

(i) *Jacobian sparsity is RAG adjacency.* Since $J_{ab} = \sum_r S_{ar} \partial J_r / \partial c_b$ and J_r depends only on the tail (precursor) vertices of r , we have $J_{ab} \neq 0$ only if some edge has b in its tail and a in its tail or head. Applied to the edge classes, this rule yields the structure sketched in Fig. S2. Unary ladder edges (growth, shrinkage, dissociation) couple n to $n \pm 1$, a tridiagonal band. Every monomer reaction has the monomer in its tail, so each monomer is a hub of degree $O(N)$ and produces a dense column. A binary edge with a mobile partner of size $m \leq i_{\text{mobile}}$ couples n to $n + m$, widening the band to $O(i_{\text{mobile}} + v_{\text{mobile}})$ and making the columns of all mobile sizes dense. Sessile–sessile pairs have zero kernel, since both diffusivities vanish, and contribute nothing. The Jacobian therefore has the bordered-banded form

$$\mathbf{J} \approx \mathbf{T} + \mathbf{U}\mathbf{V}^T, \quad b_w = \max(2i_{\text{mobile}}, 2v_{\text{mobile}}) + 1, \quad r = \text{rank } \mathbf{U} = i_{\text{mobile}} + v_{\text{mobile}}, \quad (\text{S2})$$

where $\mathbf{T}$ is banded with half-bandwidth b_w and $\mathbf{V}$ selects the mobile columns. These are precisely the defaults of the Woodbury preconditioner in Layer 3 (`prec_bw`, `prec_rank`). The band is probed with $2b_w + 1$ colored right-hand-side evaluations and factored with LAPACK, the r mobile columns are probed individually, and the Newton system $(\mathbf{I} - \gamma \mathbf{J})\mathbf{x} = \mathbf{b}$ is solved through the Sherman–Morrison–Woodbury identity [21] with an $r \times r$ Schur complement. The long-range entries in the mobile *rows* are omitted, which GMRES tolerates as a preconditioner approximation. The KLU path uses the same RAG-derived pattern explicitly (`sparse_jacobian.cpp`); the number of colors, and hence of right-hand-side evaluations per Jacobian, is the chromatic number of the column-intersection graph of that pattern.

(ii) *Stiffness is the spread of edge rates.* The fastest edges (monomer recombination and 3D/1D migration of small SIA clusters) and the slowest (monomer absorption by large sessile clusters) differ by many orders of magnitude. The system is therefore stiff, which is why Layer 3 uses implicit BDF integration with a Newton–Krylov or direct inner solver. For discrete systems with $N_{\text{eq}} \sim 10^4$, GMRES with a diagonal (Jacobi) or bordered-banded (Woodbury) preconditioner outperforms a sparse direct factorization, whose repeated colored Jacobian rebuilds dominate the cost. For bin-moment systems with $N_{\text{eq}} \sim 10^2$, the cost shifts to the per-size reconstruction inside each right-hand-side call, and the choice of preconditioner matters less.

(iii) *Reachability governs the active window.* Vertices beyond the growth front of a size ladder carry negligible concentration and are reachable only through growth or coalescence edges leaving the front. The active-window mode integrates the reachable subgraph and advances the upper index by a fixed pad whenever the concentration at the front exceeds a threshold. This is a breadth-first expansion of the RAG from the monomer roots, driven by the solution itself.

(iv) *Separating topology from kernels preserves solver structure.* Because $(\mathcal{V}, \mathcal{E})$ and k are separate objects, processes with slowly evolving kernels can be treated by operator splitting. For example, the network dislocation density in the loop-to-network loss channel is frozen within an integration segment, and the kernels are rebuilt (`rebuild_rates`) between segments without changing the edge set. The sparsity of Eq. (S2), the coloring and the preconditioner therefore carry over unchanged across segments. The graph-level reductions work in the same way. Bin moments replace a vertex range by bin super-vertices but evaluate the exact per-size edge fluxes, so conservation is inherited; they change N_{eq} and the coupling pattern, which is why the colored-KLU path is restricted to discrete modes. Because conservation of q is built into the class contracts, the solver

References

- [1] F. Horn and R. Jackson. General mass action kinetics. *Archive for Rational Mechanics and Analysis*, 47:81–116, 1972. doi: 10.1007/BF00251225.
- [2] M. Feinberg. Chemical reaction network structure and the stability of complex isothermal reactors—I. The deficiency zero and deficiency one theorems. *Chemical Engineering Science*, 42(10):2229–2268, 1987. doi: 10.1016/0009-2509(87)80099-4.
- [3] M. Feinberg. *Foundations of Chemical Reaction Network Theory*, volume 202 of *Applied Mathematical Sciences*. Springer, Cham, 2019. doi: 10.1007/978-3-030-03858-8.
- [4] P. Érdi and J. Tóth. *Mathematical Models of Chemical Reactions: Theory and Applications of Deterministic and Stochastic Models*. Manchester University Press, Manchester, 1989.
- [5] S. Klamt, U.-U. Haus, and F. Theis. Hypergraphs and cellular networks. *PLoS Computational Biology*, 5(5):e1000385, 2009. doi: 10.1371/journal.pcbi.1000385.
- [6] I. Koch. Petri nets – a mathematical formalism to analyze chemical reaction networks. *Molecular Informatics*, 29(12):838–843, 2010. doi: 10.1002/minf.201000086.
- [7] M. von Smoluchowski. Versuch einer mathematischen Theorie der Koagulationskinetik kolloider Lösungen. *Zeitschrift für Physikalische Chemie*, 92:129–168, 1917.
- [8] J. R. Faeder, M. L. Blinov, and W. S. Hlavacek. Rule-based modeling of biochemical systems with BioNetGen. In *Systems Biology*, volume 500 of *Methods in Molecular Biology*, pages 113–167. Humana Press, 2009. doi: 10.1007/978-1-59745-525-1_5.
- [9] V. Danos and C. Laneve. Formal molecular biology. *Theoretical Computer Science*, 325(1):69–110, 2004. doi: 10.1016/j.tcs.2004.03.065.
- [10] C. W. Gao, J. W. Allen, W. H. Green, and R. H. West. Reaction Mechanism Generator: Automatic construction of chemical kinetic mechanisms. *Computer Physics Communications*, 203:212–225, 2016. doi: 10.1016/j.cpc.2016.02.013.
- [11] J. Wei and J. C. W. Kuo. A lumping analysis in monomolecular reaction systems. Analysis of the exactly lumpable system. *Industrial & Engineering Chemistry Fundamentals*, 8(1):114–123, 1969. doi: 10.1021/i160029a019.
- [12] D. T. Gillespie. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of Computational Physics*, 22(4):403–434, 1976. doi: 10.1016/0021-9991(76)90041-3.
- [13] J. Marian and V. V. Bulatov. Stochastic cluster dynamics method for simulations of multi-species irradiation damage accumulation. *Journal of Nuclear Materials*, 415(1):84–95, 2011. doi: 10.1016/j.jnucmat.2011.05.045.
- [14] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, and J. Millman, editors, *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, Pasadena, CA, 2008.
- [15] B. Meyer. *Object-Oriented Software Construction*. Prentice Hall, New York, 1988.

- [16] A. C. Hindmarsh, P. N. Brown, K. E. Grant, S. L. Lee, R. Serban, D. E. Shumaker, and C. S. Woodward. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. *ACM Transactions on Mathematical Software*, 31(3):363–396, 2005. doi: 10.1145/1089014.1089020.
- [17] D. J. Gardner, D. R. Reynolds, C. S. Woodward, and C. J. Balos. Enabling new flexibility in the SUNDIALS suite of nonlinear and differential/algebraic equation solvers. *ACM Transactions on Mathematical Software*, 48(3):31, 2022. doi: 10.1145/3539801.
- [18] Y. Saad and M. H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986. doi: 10.1137/0907058.
- [19] T. A. Davis and E. Palamadai Natarajan. Algorithm 907: KLU, a direct sparse solver for circuit simulation problems. *ACM Transactions on Mathematical Software*, 37(3):36, 2010. doi: 10.1145/1824801.1824814.
- [20] A. R. Curtis, M. J. D. Powell, and J. K. Reid. On the estimation of sparse Jacobian matrices. *IMA Journal of Applied Mathematics*, 13(1):117–119, 1974. doi: 10.1093/imamat/13.1.117.
- [21] W. W. Hager. Updating the inverse of a matrix. *SIAM Review*, 31(2):221–239, 1989. doi: 10.1137/1031049.